

Homework 2: Convolutional Neural Networks

Math Section

Alexander Quispe Rojas
Universidad del Pacífico – MECA

Instructions: This homework covers the mathematical foundations of Convolutional Neural Networks (CNNs). You may discuss the problems with classmates but must write up your own solutions. Show all work for full credit. Total: **71 points**.

This homework is structured around four themes: (1) Convolution Fundamentals, (2) Pooling, (3) Architecture Analysis, (4) Backpropagation through Conv layers.

1 Convolution Fundamentals [29 Points]

Materials: Lectures 1, 2, 4 on perceptrons, backpropagation, and CNNs.

Problem 1 [5 Points]

Fully-Connected vs Convolutional Parameter Counting.

Consider an input image of size $32 \times 32 \times 3$ (e.g., a CIFAR-10 image).

- How many parameters (including biases) does a **fully-connected** layer with 128 output units have when the image is flattened?
- How many parameters does a **Conv2d** layer with 16 filters of size 5×5 have (with biases)?
- Compute the ratio of (a) to (b). What does this tell you about the parameter efficiency of CNNs?
- Explain in 2-3 sentences why this saving is achievable, in terms of *locality* and *weight sharing*.

a)
$$\text{Parameters}_{fc} = \underbrace{32 \times 32 \times 3}_{\text{image size}} \times \underbrace{128}_{\text{neurons}} + \underbrace{128}_{\text{bias}} = 393,344$$

b)
$$\text{Parameter}_{conv2d} = \underbrace{16}_{\text{out channels}} \times (\underbrace{5 \times 5}_{\text{kernel size}} \times \underbrace{3}_{\text{channels}} + \underbrace{1}_{\text{bias}}) = 1,216$$

c)
$$\text{ratio (a) to (b)} \Rightarrow \frac{\text{Parameters}_{fc}}{\text{Parameters}_{conv2d}} = \frac{393,344}{1,216} \approx 323$$
 the fc layer uses 323 times more parameters to process the same image

d) *locality*: each output neuron connects to a $5 \times 5 \times 3$ patch – not all pixels.
the kernel has 76 parameters regardless of the image's size; the fc layer scales with $H \times W$
weight sharing: the same 76 parameters filter is applied at every spatial position (slide and reuse). the fc layer requires a distinct set of weights for each of the 128 output neurons, none shared across positions

Problem 2 [8 Points]**2D Convolution by Hand.**

Consider the 5×5 input image $\mathbf{X}$ and 3×3 kernel $\mathbf{K}$:

$$\mathbf{X} = \begin{pmatrix} 1 & 2 & 3 & 0 & 1 \\ 0 & 1 & 2 & 3 & 1 \\ 2 & 1 & 0 & 1 & 2 \\ 1 & 0 & 2 & 3 & 0 \\ 0 & 1 & 1 & 2 & 1 \end{pmatrix}, \quad \mathbf{K} = \begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}$$

This is a vertical edge detector (similar to the Sobel filter).

- Compute the output $\mathbf{Y} = \mathbf{X} * \mathbf{K}$ with stride 1 and no padding. Show the dot product for each output position.
- What is the shape of $\mathbf{Y}$? Verify with the formula $W_{\text{out}} = \lfloor (W_{\text{in}} + 2P - F)/S \rfloor + 1$.
- Now compute $\mathbf{Y}$ with stride 2 and padding 1 (zero-padding). What is the new shape?

(a) Each output entry $Y[i, j] = \sum_{m=0}^2 \sum_{n=0}^2 K[m, n] \cdot X[i+m, j+n]$. The kernel zeroes the middle column and computes (left column - right column)

$$Y[0, 0] = (1 \times 1 + 0 \times 1 + 2 \times 1) + 0 \times (\text{third col}) + (3 + 2 + 0) \times -1 = -2$$

$$Y[0, 1] = (2 + 1 + 1) \times 1 + 0 \times (\text{third col}) + (0 + 3 + 1) \times -1 = 0$$

$$Y[0, 2] = (3 + 2 + 0) \times 1 + 0 \times (\text{third col}) + (1 + 1 + 2) \times -1 = 1$$

$$Y[1, 0] = (0 + 2 + 1) \times 1 + 0 \times (\text{third col}) + (2 + 0 + 2) \times -1 = -1$$

$$Y[1, 1] = (1 + 1 + 0) \times 1 + 0 \times (\text{third col}) + (3 + 1 + 3) \times -1 = -5$$

$$Y[1, 2] = (2 + 0 + 1) \times 1 + 0 \times (\text{third col}) + (1 + 2 + 0) \times -1 = 1$$

$$Y[2, 0] = (2 + 1 + 0) \times 1 + 0 \times (\text{third col}) + (0 + 2 + 1) \times -1 = 0$$

$$Y[2, 1] = (1 + 0 + 1) \times 1 + 0 \times (\text{third col}) + (1 + 3 + 2) \times -1 = -4$$

$$Y[2, 2] = (0 + 2 + 1) \times 1 + 0 \times (\text{third col}) + (2 + 0 + 1) \times -1 = 0$$

$$\mathbf{Y} = \begin{pmatrix} -2 & 0 & 1 \\ -1 & -5 & 1 \\ 0 & -4 & 0 \end{pmatrix}$$

b) Shape of Y

$$W_{out} = \left\lfloor \frac{W_{in} + 2P - F}{S} \right\rfloor + 1 = \left\lfloor \frac{5 + 0 - 3}{1} \right\rfloor + 1 = 2 + 1 = 3$$

$$Y \in \mathbb{R}^{3 \times 3}$$

c) Shape when stride 2, padding 1.

$$W_{out} = \left\lfloor \frac{5 + 2 \cdot 1 - 3}{2} \right\rfloor + 1 = \left\lfloor \frac{4}{2} \right\rfloor + 1 = 3$$

output shape : 3×3

Problem 3 [4 Points]**Output Size Formula (Multi-channel).**

Consider a Conv2d layer with input shape $H \times W \times C_{in}$ and:

- Filter size: $F \times F$
- Number of filters: C_{out}
- Stride: S
- Padding: P

- Derive the output shape $H' \times W' \times C'$.
- Derive the number of parameters in this layer (with biases).
- Apply your formulas to: input $224 \times 224 \times 3$, $F = 7$, $C_{out} = 64$, $S = 2$, $P = 3$. Give output shape and parameter count.

$$(a) \quad H' = \left\lfloor \frac{H + 2P - F}{S} \right\rfloor + 1, \quad W' = \left\lfloor \frac{W + 2P - F}{S} \right\rfloor + 1, \quad C' = C_{out}$$

(b) Parameters
each of the C_{out} filters has shape $F \times F \times C_{in}$ (weights) plus bias.
 $\Rightarrow \# \text{Parameters} = C_{out} \cdot (F^2 \cdot C_{in} + 1)$

$$(c) \quad H = 224, \quad W = 224, \quad C_{in} = 3, \quad F = 7, \quad C_{out} = 64, \quad S = 2, \quad P = 3$$

$$H' = W' = \left\lfloor \frac{224 + 6 - 7}{2} \right\rfloor + 1 = \left\lfloor \frac{223}{2} \right\rfloor + 1 = 111 + 1 = 112$$

Output shape: $112 \times 112 \times 64$

$$\text{Parameters: } 64 \cdot (7^2 \cdot 3 + 1) = 64 \cdot 148 = 9,472$$

Problem 4 [4 Points]**Hand-Designed Edge Detector.**

- (a) Construct a 3×3 kernel $\mathbf{K}_v$ that detects **vertical** edges (i.e., produces a strong activation when there is a sharp horizontal change in pixel intensity).
- (b) Apply your kernel to the following 4×4 image with a vertical edge:

$$\mathbf{X} = \begin{pmatrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \end{pmatrix}$$

Compute the output (no padding, stride 1).

- (c) Modify your kernel to detect horizontal edges instead. Show the kernel and compute the output on the same image.

(a) Vertical edge kernel K_v , the standard Sobel:

$$K_v = \begin{pmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{pmatrix}$$

response is positive where intensity increases left-to-right and negative where it decreases

(b) $X = \begin{pmatrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \end{pmatrix}$ no padding, stride = 1

$$Y[0,0] = (0 \times 1 + 0 \times 2 + 0 \times 1) + (-1 - 2 - 1) = -4$$

$$Y[0,1] = Y[0,0]$$

$$Y[1,0] = Y[0,0]$$

$$Y[1,1] = Y[0,0]$$

$$Y = \begin{pmatrix} -4 & -4 \\ -4 & -4 \end{pmatrix}$$

(c) $K_h = K_v^T = \begin{pmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{pmatrix}$

$$Y[0,0] = (0 \times 1 + 0 \times 2 + 1 \times 1) + (-1 \times 0 + -2 \times 0 + -1 \times 1) = 1 - 1 = 0 \Rightarrow Y = \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$$

$$Y[0,1] = 0, Y[1,0] = 0, Y[1,1] = 0$$

all rows are identical

Problem 5 [4 Points]**Padding Modes Comparison.**

- Define the three common padding modes: *valid*, *same*, and *full*.
- Compute the output size for each mode given an input of size W and filter of size F (assume stride 1).
- Show that “same” padding requires $P = (F - 1)/2$ for stride 1 and odd F . What happens for even F ?
- Why is “same” padding particularly useful when stacking many conv layers?

a) • Valid ($P=0$): no padding, output is strictly smaller than input
 • same ($P = (F-1)/2$ for odds F): output has the same spatial dimensions as input (with stride 1)
 • full ($P = F-1$): every input pixel is covered by the filter at least once; output is larger than input

b) output sizes for input W , filter F , stride 1

$$W_{\text{valid}} = W - F + 1, \quad W_{\text{same}} = W, \quad W_{\text{full}} = W + F - 1$$

c) formula $S=1$

$$W_{\text{out}} = \left\lfloor \frac{W + 2P - F}{S} \right\rfloor + 1 = W + 2P - F + 1$$

$$\text{if } W_{\text{out}} = W \Rightarrow W = W + 2P - F + 1 \Rightarrow P = (F-1)/2$$

for even F , $(F-1)/2$ is not an integer, so exact symmetric padding is impossible. Standard practice is to use odd kernel size (3, 5, 7, ...)

d) without same padding, each conv. layer with kernel F shrinks the map by $F-1$ pixels per side. After L layers of 3×3 convs., a 224×224 input shrinks by $2L$ pixels. Same padding allows arbitrarily deep stacks with no spatial shrinkage from convolution, giving the designer full control over when down sampling occurs (via stride or explicit pooling)

Problem 6 [4 Points]**Receptive Field Through a Sample Network.**

Compute the receptive field at each layer of the following network applied to a 28×28 image.

Use the recurrences:

$$j_l = j_{l-1} \cdot S_l, \quad \text{RF}_l = \text{RF}_{l-1} + (F_l - 1) \cdot j_{l-1}$$

with initial conditions $j_0 = 1, \text{RF}_0 = 1$.

l	Operation	F_l	S_l	Output size
0	Input	—	—	28×28
1	Conv 5×5	5	1	?
2	MaxPool 2×2	2	2	?
3	Conv 5×5	5	1	?
4	MaxPool 2×2	2	2	?
5	Conv 3×3	3	1	?

Compute output size, j_l , and RF_l at each layer.

l	Operation	F_l	S_l	j_l	RF_l	Output size
0	Input	—	—	1	1	28×28
1	Conv 5×5	5	1	1	$1 + (5-1) \cdot 1 = 5$	$(28-5)+1 = 24 \times 24$
2	MaxPool 2×2	2	2	2	$5 + (2-1) \cdot 1 = 6$	$(24-2)/2+1 = 12 \times 12$
3	Conv 5×5	5	1	2	$6 + (5-1) \cdot 2 = 14$	$(12-5)+1 = 8 \times 8$
4	MaxPool 2×2	2	2	4	$14 + (2-1) \cdot 2 = 16$	$(8-2)/2+1 = 4 \times 4$
5	Conv 3×3	3	1	4	$16 + (3-1) \cdot 4 = 24$	$(4-3)+1 = 2 \times 2$

2 Pooling [8 Points]*Materials: Lecture 4 on CNNs.***Problem 7 [4 Points]****Max vs Average Pooling.**Consider the 4×4 feature map:

$$\mathbf{X} = \begin{pmatrix} 2 & 5 & 1 & 3 \\ 4 & 1 & 8 & 0 \\ 7 & 2 & 1 & 4 \\ 3 & 0 & 6 & 9 \end{pmatrix}$$

- (a) Apply 2×2 **max pooling** with stride 2.
 (b) Apply 2×2 **average pooling** with stride 2.
 (c) Discuss: when would you prefer max over average pooling?

(a) $Y[0,0] = \max\{2, 5, 1, 3\} = 5$
 $Y[0,1] = \max\{4, 1, 8, 0\} = 8$
 $Y[1,0] = 7$
 $Y[1,1] = 9$
 $Y_{\max} = \begin{pmatrix} 5 & 8 \\ 7 & 9 \end{pmatrix}$

(b) $Y[0,0] = (2+5+1+3)/4 = 3$
 $Y[0,1] = (4+1+8+0)/4 = 3$
 $Y[1,0] = (7+2+1+4)/4 = 3$
 $Y[1,1] = (3+0+6+9)/4 = 5$
 $Y_{\text{avg}} = \begin{pmatrix} 3 & 3 \\ 3 & 5 \end{pmatrix}$

- (c) • Max pooling should be preferred when the presence of a feature matters more than its average strength – typical in classification and detection tasks.
 • Average pooling is preferred for global aggregation, where retaining a summary of the entire spatial activation map matters more than preserving peak responses.

Problem 8 [4 Points]**Pooling Effect on Receptive Field.**

Consider a network with the following layers (all using 3×3 kernels, stride 1, no padding):

$$\text{Conv} \rightarrow \text{Conv} \rightarrow \text{MaxPool}(2 \times 2, \text{stride } 2) \rightarrow \text{Conv} \rightarrow \text{Conv}$$

- (a) What is the receptive field after the first two conv layers?
 (b) What is the jump factor j after the maxpool layer?
 (c) What is the receptive field after the final conv layer?

l	Operation	f	s	j_t	Rf_t
0	input	-	-	1	1
1	Conv 3×3	3	1	1	$1 + 2 \times 1 = 3$
2	Conv 3×3	3	1	1	$3 + 2 \times 1 = 5$
3	Maxpool 2×2	2	2	2	$5 + 1 \times 1 = 6$
4	Conv 3×3	3	1	2	$6 + 2 \times 2 = 10$
5	Conv 3×3	3	1	2	$10 + 2 \times 2 = 14$

(a) After the first two conv layers: $Rf = 5$

(b) jump factor j after the maxpool layer: $j = 2$

(c) After the final conv. layer: $Rf = 14$

3 Architecture Analysis [14 Points]

Materials: Lecture 4: classic CNN architectures (LeNet, VGG).

Problem 9 [7 Points]**LeNet-5 Parameter Count.**

The original LeNet-5 (LeCun et al., 1998) has the following architecture for 32×32 grayscale input:

Layer	Operation	Output size	Parameters
1	Conv 5×5 , 6 filters	$28 \times 28 \times 6$	?
2	AvgPool 2×2 , stride 2	$14 \times 14 \times 6$	0
3	Conv 5×5 , 16 filters	$10 \times 10 \times 16$	?
4	AvgPool 2×2 , stride 2	$5 \times 5 \times 16$	0
5	FC $\rightarrow 120$	120	?
6	FC $\rightarrow 84$	84	?
7	FC $\rightarrow 10$	10	?

- (a) Compute the parameter count for each layer (with biases).
 (b) What is the total parameter count?
 (c) Compare with an MLP that has 2 hidden layers of 300 units each. Which has more parameters?

(a)

Layer	Operation	Output size
1	Conv 5×5 , 6 filters	$28 \times 28 \times 6$
2	AvgPool 2×2 , stride 2	$14 \times 14 \times 6$
3	Conv 5×5 , 16 filters	$10 \times 10 \times 16$
4	AvgPool 2×2 , stride 2	$5 \times 5 \times 16$
5	FC $\rightarrow 120$	120
6	FC $\rightarrow 84$	84
7	FC $\rightarrow 10$	10

Parameters

$$6(5^2 \cdot 1 + 1) = 6 \times 26 = 156$$

0 (no learned parameters)

$$16(5^2 \cdot 6 + 1) = 16 \times 151 = 2,416$$

$$5 \times 5 \times 16 \times 120 + 120 = 48,120$$

$$120 \times 84 + 84 = 10,164$$

$$10 \times 84 + 10 = 850$$

(b) total parameters = $156 + 0 + 2,416 + 0 + 48,120 + 10,164 + 850$
 $= 61,706$

(c) compare with MLP

$$\text{input to MLP: } 32 \times 32 = 1,024$$

$$\text{FC}_1: 1024 \times 300 + 300 = 307,500$$

$$\text{FC}_2: 300 \times 300 + 300 = 90,300$$

$$\text{FC}_3: 300 \times 10 + 10 = 3,010$$

$$\text{total MLP} = 307,500 + 90,300 + 3,010 = 400,810$$

the MLP has 400,810 parameters — approximately 6.5x more than LeNet-5.

Problem 10 [7 Points]**Why 3×3 Filters Dominate.**

In modern CNNs (VGG, ResNet, etc.), the 3×3 filter has become standard. Compare two architectures with the same effective receptive field of 5×5 :

- **Architecture A:** One Conv 5×5 layer with C input and C output channels.
- **Architecture B:** Two stacked Conv 3×3 layers (each $C \rightarrow C$).

- Compute the receptive field of each architecture (verify both are 5×5).
- Compute the parameter count of each (ignore biases for simplicity).
- Count the number of nonlinearities (ReLU layers) each can apply.
- Discuss: why is Architecture B preferred?

(a) Receptive field of each architecture:

• Architecture A (one 5×5 conv) $\Rightarrow$ RF = 5 directly (one layer)

• Architecture B

$$RF_1 = 3 \text{ (directly first layer)}$$

$$RF_2 = RF_1 + (F_2 - 1) \cdot j_{t-1}$$

$$= 3 + (3 - 1) \cdot 1 = 5$$

Both architectures have effective receptive field 5×5

(b) ignoring bias

$$A: 5^2(C \cdot C) = 25C^2$$

$$B: 3^2(C \cdot C) \times 2 = 18C^2$$

Architecture B uses $\frac{72}{100}$ of A's parameters, a 28% reduction for the same receptive field

(c) numbers of nonlinearities:

A: 1 ReLU (after the single conv)

B: 2 ReLU (one between the two conv, one after the second)

(d) Architecture B achieves the same receptive field with fewer parameters and more nonlinearities. Each additional ReLU adds a "kink" in the network's learned function, increasing the class of functions representable at the same computational cost. This double advantage (fewer parameters, more expressive) makes stacked 3×3 convolutions the standard building block of all major modern CNN architectures.

4 Backpropagation Through Conv Layers [20 Points]

Materials: Lecture 2 on backpropagation, Lecture 4 on CNNs.

Problem 11 [10 Points]**Gradient w.r.t. the Convolution Kernel.**

Consider a 1D convolution (single channel) with input $\mathbf{x} \in \mathbb{R}^n$, kernel $\mathbf{K} \in \mathbb{R}^k$, and output $\mathbf{y} \in \mathbb{R}^{n-k+1}$:

$$y_i = \sum_{u=0}^{k-1} x_{i+u} K_u, \quad i = 0, 1, \dots, n-k$$

Let $\mathcal{L}$ be a downstream loss. Define $\delta_i \equiv \partial \mathcal{L} / \partial y_i$ (the upstream gradient).

- (a) Derive a formula for $\partial \mathcal{L} / \partial K_u$.
- (b) Derive a formula for $\partial \mathcal{L} / \partial x_j$.
- (c) Show that the gradient w.r.t. the kernel is itself a **convolution** between the input $\mathbf{x}$ and the upstream gradient $\boldsymbol{\delta}$.
- (d) Show that the gradient w.r.t. the input is a **full convolution** between $\boldsymbol{\delta}$ and the **flipped** kernel.

Problem 12 [5 Points]

Gradient Through Max Pooling.

Consider a 2×2 max pooling layer with stride 2. Let the output at position (i, j) be:

$$y_{i,j} = \max_{(u,v) \in W_{i,j}} x_{u,v}$$

where $W_{i,j}$ is the 2×2 window for output (i, j) .

- (a) Derive $\partial \mathcal{L} / \partial x_{u,v}$ in terms of the upstream gradient $\delta_{i,j} = \partial \mathcal{L} / \partial y_{i,j}$.
- (b) Apply your formula: given input

$$\mathbf{X} = \begin{pmatrix} 1 & 3 & 2 & 0 \\ 5 & 4 & 1 & 6 \\ 0 & 2 & 7 & 3 \\ 1 & 1 & 5 & 8 \end{pmatrix}, \quad \boldsymbol{\delta} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

Compute $\partial \mathcal{L} / \partial \mathbf{X}$ entry-by-entry.

- (c) Why is the gradient sparse?

Problem 13 [5 Points]

Gradient Through Average Pooling.

Repeat the previous problem for 2×2 average pooling with stride 2:

$$y_{i,j} = \frac{1}{4} \sum_{(u,v) \in W_{i,j}} x_{u,v}$$

- (a) Derive $\partial \mathcal{L} / \partial x_{u,v}$ in terms of $\delta_{i,j}$.
- (b) Apply to the same input and upstream gradient as before. Compute $\partial \mathcal{L} / \partial \mathbf{X}$.
- (c) Compare with max pool: which is more “democratic”?

Summary of Topics Covered

This homework covered:

- **Fundamentals:** parameter counting, convolution by hand, padding modes, edge detectors.
- **Pooling:** max vs avg pooling, effect on receptive field.
- **Architecture:** receptive field, LeNet parameter accounting, why 3×3 filters dominate.
- **Backpropagation:** gradient w.r.t. kernel and input, max/avg pool gradients.

Recommended reading:

- Goodfellow et al., *Deep Learning*, Chapter 9 (CNNs).
- He et al. (2015), *Deep Residual Learning for Image Recognition*.
- LeCun et al. (1998), *Gradient-Based Learning Applied to Document Recognition* (LeNet-5).
- Krizhevsky, Sutskever, Hinton (2012), *ImageNet Classification with Deep Convolutional Neural Networks* (AlexNet).

